Research Guide and Section Instructions

Resilient Data Science Programming for Fault-Tolerant Distributed Analytics in Multi-Cluster Spark Environments

Team ID	26FSPDSCPMID4098
Members	26MML0054, 26MML0006, 26MML0047, 26MML0015 (PoC)
Course	Programming for Data Science (MACSE502), SCOPE, VIT
Guide	Dr. M. Rajasekhara Babu
Based on	Section planning table 26FSPDSCPMID4098.pdf, Manuscript Guidelines, and the RMC-Spark implementation
Companion files	paper/RMC-Spark_Manuscript.docx (manuscript), paper/Diagrams/ (figures), rmc-spark/README.md (implementation)
Date	October 2026

How to use this file. The Project Overview explains the project in plain words. Part A summarises what existing research already does and where it stops. Part B states the gap. Part C explains how our system, RMC-Spark, solves it. Part D is the instruction sheet for every section of the plan: what to write, which references to cite, which figures and tables are required, the word target, the owner and what is still missing. Parts E and F are checklists and the step-by-step plan to finish. Reference numbers [n] are the same as in the manuscript.

Project Overview — What Is This Project? 3
The problem 3
Our solution: RMC-Spark 3
How we prove it 3
Part A — Existing Research (State of the Art) 4
Fault tolerance inside the Spark engine 4
Checkpoint interval optimisation 4
Fault-recovery benchmarks 4
Stragglers and gray failures 4
Failure prediction and AIOps 4
Orchestration across clusters and clouds 5
Cloud fault tolerance and resilience engineering 5
Comparative summary (Table 3 of the manuscript) 5
Part B — The Research Gap and Problem 6
Part C — How We Solve It: RMC-Spark 7
Components and how each one closes a gap 7
What is new compared with the closest work 7

Hypotheses the experiments must confirm or refute 7
Part D — Section-by-Section Instructions (from the planning table) 9
Executive Summary and Abstract 9
Keywords 9
1. Introduction 9
2. Problem Statement 10
3. Background and Literature Review 10
4. Requirements Analysis for Resilient Analytics Systems 10
5. Multi-Cluster Spark Analytics Architecture 11
6. Distributed Data Engineering Layer 11
7. Spark Processing and Execution Layer 11
8. Fault Detection and Monitoring Framework 12
9. Fault-Tolerant Recovery and Resilience Layer 12
10. Multi-Cluster Coordination and Orchestration Layer 13
11. Explainability and Reliability Assessment Layer 13
12. Security, Governance, and Compliance Layer 13
13. Experimental Setup 14
14. Evaluation Metrics 14
15. Results and Findings 14
16. Comparative Analysis 15
17. Reliability and Business Continuity Assessment 15
18. Industry Use Cases and Case Study Evaluation 15
19. Business Impact Assessment 16
20. Challenges and Limitations 16
21. Future Research Directions 16
22. Conclusion 17
References 17
Appendix (Optional) 17
Part E — Compliance Checklists 18
Publication statistics (page 5 of the plan) 18
Core figures and tables 18
Manuscript guidelines 18
Part F — Step-by-Step Plan to Finish 19
Team responsibilities 19

Project Overview — What Is This Project?

This project is a research paper plus a working system for **Programming for Data Science (MACSE502)** at VIT (Team 26FSPDSCPMID4098, guide Dr. M. Rajasekhara Babu).

Title: Resilient Data Science Programming for Fault-Tolerant Distributed Analytics in Multi-Cluster Spark Environments

The problem

Apache Spark is the standard engine for big-data analytics. If a single machine or task fails, Spark recovers on its own. But if the whole cluster goes down (master crash, network split, data-centre outage), every job on it dies and someone has to rerun it from the start. Long pipelines, such as ML training, can lose hours of work, and today's tools rarely explain why they acted.

Our solution: RMC-Spark

A layer that sits above two or more ordinary Spark clusters:

1. **Health monitor:** checks every cluster every 2 seconds and gives it a health score. A cluster is marked FAILED only on strong evidence.
2. **Resumable pipeline stages (the rds library):** each step of a PySpark pipeline saves its result to shared storage. The timing follows a checkpoint formula (Young/Daly), so the work lost in a failure stays small.
3. **Smart failover:** if a cluster dies and the job has stopped making progress, the job moves to the healthy cluster. There it continues from the last saved step instead of starting over.
4. **Placement:** picks the best cluster from its load and health.
5. **Explainability:** every decision is logged with its reason, and each cluster gets a reliability score.

In one line: when a Spark cluster crashes, the job moves to another cluster and carries on from where it stopped, and the system explains why.

How we prove it

The whole system runs on a laptop in Docker: 2 Spark clusters, MinIO storage, Prometheus/Grafana, and the controller. We deliberately break it with 9 kinds of faults, such as killing a cluster or slowing the network. Then we compare five setups, from plain Spark through simple failover to RMC-Spark, on four workloads: ETL, machine learning, iterative PageRank and streaming.

Part A — Existing Research (State of the Art)

The literature used in the manuscript (127 references, 108 from 2020–2025, 114 journal papers) falls into seven themes. For each theme the table lists representative work, what it achieves, and what it does not solve for our problem.

Fault tolerance inside the Spark engine

Representative work	[5] RDD lineage (Zaharia 2012); [39] RCMP (Dinu 2014); [23] Lineage checkpoint (Kweun 2020); [27] Write-ahead lineage (Wang 2024); [40] cache recovery (Li 2021); [41] LT-code caching (2024); [42] GraphX fault tolerance (He 2025); [6] systematic survey (Isukapalli 2024); [4] Spark ecosystem survey (Tang 2020)
What it does well	Cheap, transparent recovery of lost partitions and tasks; techniques to shorten long lineage.
What is missing for our problem	All assume the cluster survives. None resumes a job on a different cluster after an outage.

Checkpoint interval optimisation

Representative work	[43] Young 1974; [44] Daly 2006; [45] En-CHORE online MTBF (Sigdel 2020); [46][47] utilisation-optimal stream checkpoints (Jayasekara 2020/2021); [48] Flink interval (Zhang 2021); [49] FATM failure-aware (Akber 2021); [50] incremental checkpointing; [51] convergence-aware (Li 2024)
What it does well	Solid theory for when to checkpoint given checkpoint cost C and MTBF M; adaptive variants exist.
What is missing for our problem	Applied to HPC jobs or stream operators, not to data science pipeline stages; MTBF not shared across clusters.

Fault-recovery benchmarks

Representative work	[52] van Dongen 2021 (IEEE Access, 4 stream frameworks); [53] Vogel 2024 (ACM DEBS, Kubernetes + chaos); [54] PGVal (Tahir 2024, PVLDB); [19] stream processing survey (Fragkoulis 2023)
What it does well	Rigorous measurement of downtime, recovery time and data loss under injected faults.
What is missing for our problem	Single-cluster only; whole-cluster loss not tested; no comparison of recovery strategies on the same engine.

Stragglers and gray failures

Representative work	[55] LATE (2008); [56] Mantri (2010); [57] Hawkeye RL (Du 2020); [58] ANN stragglers (2020); [34] straggler survey (Bawankule 2022); [59] Hone (Li 2021); [24] Gray Failure (Huang 2017)
What it does well	Speculation and learned straggler identification; the gray-failure concept explains silent degradation.
What is missing for our problem	Not combined with cluster-level failover; most detectors look only at one side (cluster or job).

Failure prediction and AIOps

Representative work	[35] Bi-LSTM task failure (Gao 2020); [60] ML vs DL predictors (Asmawi 2022); [61] ML job failures (2023); [62] proactive failure-aware scheduling (Alahmad 2021); [63] OFP-TM; [30][31] node failure prediction; [32][33] disk failure; [64] workload prediction; [65] AIOps survey (Notaro 2021); [66][67] MLOps/LLM AIOps; [68]–[77] anomaly detection & root-cause analysis
What it does well	Accurate prediction and detection on large traces; rich anomaly/RCA techniques.

What is missing for our problem	Outputs rarely trigger concrete recovery of analytics jobs; models are opaque — operators need interpretable decisions [30].

Orchestration across clusters and clouds

Representative work	[13] Borg; [14] Borg/Omega/Kubernetes; [78]–[80] Kubernetes scheduling surveys; [81][82] container orchestration; [83] HPC containers; [25] multi-cloud orchestration review; [84] multi-cloud scheduling (Zhu 2021); [85]–[91] geo-distributed analytics; [28] RL multi-cloud Spark scheduler with micro-checkpointing (Raj 2026) — CLOSEST WORK
What it does well	Placement, scheduling and data movement across sites and clouds; pod restarts inside one Kubernetes cluster.
What is missing for our problem	Little support for continuing a running analytics job after its cluster disappears; learned schedulers explain little.

Cloud fault tolerance and resilience engineering

Representative work	[92]–[94] proactive FT, reliability & availability models; [36][37][22] FT surveys; [95] Chaos Engineering (Basiri 2016); [96]–[99] chaos for security, Docker, digital twins, MLR 2025; [29][38] SLO/SLA
What it does well	Availability modelling and controlled fault experiments are mature practice.
What is missing for our problem	Not specialised to resumable analytics pipelines; rarely produce reproducible, open testbeds for Spark.

Comparative summary (Table 3 of the manuscript)

Approach	Cross-cluster recovery	Adaptive checkpoint	Gray-failure aware	Explainable	Fault injection
RDD lineage [5]	No	No	No	n/a	No
RCMP [39]	No	Partial	No	n/a	Yes
Lineage checkpoint [23]	No	No	No	n/a	Yes
Write-ahead lineage [27]	No	No	No	n/a	Yes
En-CHORE [45]	No	Yes	No	n/a	Simulated
Optimal stream checkpoint [47]	No	Yes	No	n/a	Yes
FATM [49]	No	Yes	No	n/a	Yes
Hawkeye [57]	No	No	Yes	Low	No
Bi-LSTM prediction [35]	No	No	Partial	Low	Trace
Proactive scheduling [62]	No	No	Partial	Low	Trace
Recovery benchmark [53]	No	No	No	n/a	Yes
Multi-cloud RL scheduler [28]	Yes	Yes	Partial	Low	Not stated
RMC-Spark (ours)	Yes	Yes	Yes	Yes	Yes

Part B — The Research Gap and Problem

Research question. How can a data science pipeline running on one of several independent Spark clusters continue after a failure of its cluster, with bounded lost work, without restarting jobs unnecessarily, and with decisions that operators can understand and audit?

Challenge	State of practice (existing research)	Gap we address
Cluster outage	Job fails; manual or scripted restart from scratch	Automatic failover with stage-level resume
Long lineage	Recompute from source, or checkpoint everything	Checkpoint interval derived from the observed failure rate
Gray failures	Usually invisible to health checks	Latency and job-progress signals inside the health score
Unnecessary restarts	Timeout-based HA restarts healthy jobs	Migrate only jobs that stopped progressing
Opaque decisions	Learned or scripted policies without rationale	Per-signal contributions logged for every decision
Reproducible evaluation	Ad hoc fault experiments, single cluster	Scripted fault injection, five configurations, statistics

Formal view (Section II). A job J is a sequence of stages s1…sn with compute times c1…cn on one of K clusters; cluster k fails with mean time between failures Mk. Objectives: (1) maximise the probability that J finishes before its deadline, (2) minimise expected completion time including recovery, (3) keep fault-free overhead small, (4) justify every recovery action with observable evidence.

Part C — How We Solve It: RMC-Spark

RMC-Spark is a resilience layer above several unmodified Spark clusters. Two ideas make it work: (1) Spark drivers run outside the clusters (client mode in a controller container), so an outage cannot kill the driver; (2) every pipeline stage commits its output to shared object storage, so a new attempt on another cluster restores finished stages instead of recomputing them.

Components and how each one closes a gap

Component	Gap it closes	How it works	Where (code / evidence)
C1 Health monitor	Gray failures; false alarms	Probes masters (/json) and drivers (REST) every 2 s; 5 signals; EWMA z-score (Eq. 2); H = 1 − Σ wi ai (Eq. 3); HEALTHY → SUSPECT → FAILED with hysteresis (3 polls < 0.50 to fail, 5 polls ≥ 0.95 to recover)	controller/monitor.py, anomaly.py, health.py, statemachine.py; tests/test_detection.py
C2 Adaptive checkpointing (rds)	Long lineage; lost work	@pipe.stage decorator; data written first, _RDS_COMMIT marker last; checkpoint when unsaved work W ≥ τ* = Daly(C, M) (Eq. 4); MTBF estimated online (Eq. 6)	rds/policy.py, rds/stage.py; tests/test_policy.py
C3 Progress-aware failover	Cluster outage; unnecessary restarts	Crash on healthy cluster → retry there; cluster FAILED and job stalled ≥ 10 s → kill driver, resubmit elsewhere with the same job id; FAILED but progressing → keep (logged)	controller/jobs.py, core.py; tests/test_controller_sim.py (outage simulation)
C4 Workload placement	Balancing; where to fail over	J = 0.4·load + 0.5·(1 − H) + 0.1·transfer (Eq. 7); failed/excluded clusters skipped; deterministic ties	controller/orchestrator.py
C5 Explainable reliability	Opaque decisions; audit	Decision log with every transition/failover/keep and its causes; reliability score R = Σ wj sj (Eq. 8)	controller/decisions.py, reliability.py, api.py
Evaluation harness	Reproducible evaluation	9 fault types via Docker API; 5 systems × 4 workloads × 8 faults × 3 reps = 480 runs; Mann-Whitney U, Cliff's δ, bootstrap CI	chaos/inject.py, experiments/, analysis/

What is new compared with the closest work

- **vs. lineage checkpointing / write-ahead lineage [23][27]:** they shorten recovery inside one cluster; we resume on another cluster and choose checkpoints from the measured failure rate.
- **vs. adaptive checkpoint theory [45][47][49]:** same Young/Daly foundation, but applied to data science pipeline stages with an MTBF shared across clusters.
- **vs. recovery benchmarks [52][53]:** we reuse their metric definitions (throughput recovery ≥ 85 % for 40 s) but add whole-cluster outages and compare recovery strategies on the same engine.
- **vs. multi-cloud RL scheduling with micro-checkpointing [28] (closest work):** we use an analytical, explainable policy instead of a learned one, record the cause of every decision, and evaluate with real injected faults on an open testbed.
- **vs. timeout-based HA scripts (baseline B3):** we distinguish a dead master with working executors from a dead cluster and never restart a job that is still progressing.

Hypotheses the experiments must confirm or refute

#	Hypothesis	Metric / evidence
H1	Under cluster outages and partitions, P completes significantly more jobs than B1 and B2	Success rate (Table 11/12)
H2	For faults needing failover, P recovers faster than B3 (health detector + stall rule beat a 30 s timeout)	Detection and recovery time, Mann-Whitney p, Cliff's δ
H3	Under master-only failures, P completes jobs without migration, B3 restarts them	keep decisions, attempts per run
H4	On the long-lineage workload W3, P and A1 recompute less work than B1–B3	Recomputed work (stage events)

#	Hypothesis	Metric / evidence
H5	In fault-free runs, P has lower overhead than A1 and B2	Overhead O (Eq. 11)

Honest limits to state (Section XX): single host; controller and MinIO are single points of failure in the testbed; at-least-once stage execution; thresholds set by reasoning, not tuned; three repetitions per cell. Published benchmark numbers come from other hardware and must not be placed in our result tables.

Part D — Section-by-Section Instructions (from the planning table)

For each section: the plan's content focus and required visuals, the word target from the plan, the current word count of the manuscript, the owner, what to write, which references to cite, where the evidence comes from, and what is still to do. Figure files are in paper/Diagrams/ as "Fig NN <Title>.png".

Executive Summary and Abstract

Plan: content focus	Summarize the challenges of distributed analytics resilience, proposed fault-tolerant Spark architecture, experimental outcomes, and industrial significance.
Plan: visuals	Table 1: Executive Summary of Contributions and Outcomes
Word count	Target 450–750 | current 490
Owner	26MML0015
What to write	1. Abstract: one paragraph, 150–250 words, no citations or equations (IEEE rule): problem → RMC-Spark (C1–C5 + rds) → implementation → evaluation → significance. 2. Executive summary: why Spark's single-cluster fault tolerance is not enough; what the rds library and controller do; Table 1 with status of each contribution. 3. After experiments: replace the generic 'measured results are reported in Section XV' sentence with the 2–3 headline numbers.
Cite	[1]–[6] (fraud/maintenance motivation, Spark engine, survey of fault-tolerant analytics)
Our evidence / code	Contributions C1–C5, rds library, test suite (56 tests), evaluation harness.
Status / to do	Insert headline measured results.

Keywords

Plan: content focus	Distributed Analytics, Apache Spark, Fault Tolerance, Multi-Cluster Computing, Resilient Data Science, High Availability, Data Engineering, Cloud Analytics.
Plan: visuals	No visuals required
Word count	Target 20–40 | current 17
Owner	26MML0054
What to write	1. Use exactly the 8 keywords of the plan, in alphabetical order (IEEE 'INDEX TERMS').
Cite	—
Our evidence / code	—
Status / to do	Done.

1. Introduction

Plan: content focus	Evolution of distributed analytics, Spark ecosystems, cloud-native data science, fault tolerance requirements, motivation for resilient analytics programming.
Plan: visuals	Figure 1: Evolution of Distributed Analytics Platforms Figure 2: Resilient Data Science Ecosystem
Word count	Target 800–1200 | current 867
Owner	26MML0047
What to write	1. A. Evolution: MapReduce → YARN → RDD lineage → Spark SQL/Streaming/Flink → Kubernetes, serverless, lakehouse (Fig. 1). 2. B. Ecosystem: programs + platform + storage + the three teams who build/operate them (Fig. 2). 3. C. Why fault tolerance got harder: long lineage, gray failures, several clusters per organisation. 4. D. Motivation and numbered contributions (1)–(5); paper organisation paragraph.
Cite	[3]–[27] incl. MapReduce [7], YARN [8], RDD [5], Spark [3], Flink [11], Borg/Kubernetes [13][14], gray failure [24], multi-cloud [25]

Our evidence / code	Contribution list = the five components + rds + open testbed.
Status / to do	Done (meets minimum).

2. Problem Statement

Plan: content focus	Node failures, cluster outages, workload imbalance, data loss, performance degradation, operational downtime.
Plan: visuals	Figure 3: Failure Scenarios in Distributed Analytics Table 2: Research Challenges and Gaps
Word count	Target 400–700 | current 501
Owner	26MML0006
What to write	1. Classify failures by blast radius: task, node, gray, cluster (Fig. 3) and who reacts first in standard Spark. 2. List the five concrete problems from the plan (outage, imbalance, data loss, degradation, downtime). 3. Formal model: job = stages s1..sn, cluster MTBF Mk; objectives (completion probability, expected time, overhead, explainability). 4. One-sentence research question + Table 2 (challenge / state of practice / gap).
Cite	[5][24][30]–[38] (node/disk failure prediction, straggler survey, lineage, SLOs)
Our evidence / code	Each gap in Table 2 maps to a component (see Part D of this guide).
Status / to do	Done.

3. Background and Literature Review

Plan: content focus	Spark fault tolerance, distributed systems resilience, cluster orchestration, data replication, workload recovery, high-availability frameworks.
Plan: visuals	Figure 4: Literature Taxonomy Framework Table 3: Comparative Analysis of Existing Approaches (core table)
Word count	Target 1500–2500 | current 1491
Owner	26MML0047
What to write	1. Organise by the 6 branches of Fig. 4: engine-level recovery, checkpoint optimisation, recovery benchmarks, stragglers & gray failures, prediction & AIOps, orchestration across clusters/clouds, resilience engineering; plus data platforms and a synthesis. 2. For each paper: one sentence on what it does + one on its limitation relative to our problem. Summarise in your own words (similarity ≤ 10 %). 3. Table 3: 13 approaches × properties (cross-cluster recovery, adaptive checkpoint, gray-failure aware, explainable, fault injection, engine). 4. End with the positioning paragraph: no reviewed approach covers all five properties.
Cite	97 references cited here; key ones: [5][23][27][39]–[49][52]–[59][28][62][65][76][78][90][95]
Our evidence / code	RMC-Spark row of Table 3 = Yes on all five properties.
Status / to do	About 10 words short of the minimum — add one sentence when you revise.

4. Requirements Analysis for Resilient Analytics Systems

Plan: content focus	Scalability, availability, recoverability, reliability, interoperability, governance, operational continuity requirements.
Plan: visuals	Figure 5: Requirements Engineering Framework Table 4: Functional and Non-Functional Requirements Matrix
Word count	Target 600–1000 | current 547
Owner	26MML0006

What to write	1. Fig. 5: stakeholders → requirements → components → verification (traceability). 2. Sub-sections: Scalability & availability; Recoverability & reliability; Interoperability, governance & continuity. 3. Table 4: F1–F7 functional, N1–N5 non-functional, each with component and verification method.
Cite	[29][38][109][110][112][113]
Our evidence / code	Every requirement is verified by a unit test, the simulation, or an experiment.
Status / to do	About 50 words short — add a short paragraph on requirement prioritisation (MoSCoW) if needed.

5. Multi-Cluster Spark Analytics Architecture

Plan: content focus	Overall architecture integrating multiple Spark clusters, data engineering pipelines, monitoring services, failover mechanisms, orchestration components.
Plan: visuals	Figure 6: Multi-Cluster Spark Architecture (CORE FIGURE) Figure 7: Layered Analytics Stack
Word count	Target 1000–1500 | current 963
Owner	26MML0015
What to write	1. Four design principles: unmodified engine; jobs separated from clusters; act on evidence of harm; explain every action. 2. Fig. 6 walkthrough: clusters A/B, MinIO, Prometheus/Grafana, controller hosting the drivers (client mode) — explain why driver placement is the key decision. 3. Component interactions + the outage sequence in words; layered stack (Fig. 7); failure domains & assumptions; deployment model (testbed vs production).
Cite	[24][25][30][65][90] + observability [99][100]
Our evidence / code	docker-compose.yml, controller/core.py; README §1.
Status / to do	Slightly below minimum; fine.

6. Distributed Data Engineering Layer

Plan: content focus	Ingestion, transformation, metadata management, schema evolution, data partitioning, storage management across clusters.
Plan: visuals	Figure 8: Distributed Data Engineering Pipeline Table 5: Data Engineering Components and Responsibilities
Word count	Target 1000–1500 | current 980
Owner	26MML0006
What to write	1. Buckets data / checkpoints / results; synthetic generator (seeded) + public datasets (NYC TLC, AI4I 2020, ULB credit card). 2. Resilient stages and the commit-marker protocol (data first, marker last; never read half-written output). 3. Metadata (markers with sequence numbers, linear-chain skipping), schema evolution policy, streaming path, partitioning (16 partitions, AQE), data quality.
Cite	[17][18][104]–[113]
Our evidence / code	rds/stage.py, jobs/datagen.py, jobs/etl_taxi.py, datasets/README.md
Status / to do	Done.

7. Spark Processing and Execution Layer

Plan: content focus	Spark execution model, RDD lineage, DAG scheduling, task execution, resource allocation, distributed processing flow.
Plan: visuals	Figure 9: Spark Execution Architecture Figure 10: DAG Processing Workflow

	Equation: Resource Utilization Function (Eq. 1)
Word count	Target 1200–1800 | current 943
Owner	26MML0006
What to write	1. Standalone master/worker model; driver in client mode in the controller; resource settings (512 MB driver, 1 core / 576 MB executors, 4 cores per job). 2. Narrow vs wide dependencies, stage boundaries, how a committed resilient stage cuts the lineage (Fig. 10). 3. Built-in fault tolerance (task.maxFailures 4, 60 s network timeout, speculation in P); Eq. 1 utilisation U; processing flow; interaction of built-in and external recovery.
Cite	[3][5][9][19][23][55][101][103]
Our evidence / code	docker/spark-defaults.conf, controller/jobs.py (spark-submit), analysis/metrics.py (U)
Status / to do	About 250 words short — after experiments, add a paragraph describing the real DAG/stage timings from the Spark UI.

8. Fault Detection and Monitoring Framework

Plan: content focus	Cluster monitoring, anomaly detection, health assessment, workload diagnostics, early-failure detection.
Plan: visuals	Figure 11: Monitoring and Observability Architecture Figure 12: Fault Detection Workflow Table 6: Monitoring Metrics and Thresholds (CORE TABLE)
Word count	Target 1000–1500 | current 1003
Owner	26MML0054
What to write	1. Two observation paths: direct probes for decisions (2 s), Prometheus/Grafana for operators (5 s). 2. Five signals; EWMA z-score (Eq. 2); additive health score H (Eq. 3) with weights; 'zero workers' override. 3. State machine HEALTHY/SUSPECT/FAILED with hysteresis (Fig. 12); why delay or stall alone only gives SUSPECT; calibration and false alarms. 4. Table 6 must contain the exact values from configs/clusters.yaml.
Cite	[24][30][35][65][67][68][73][99][100][102]
Our evidence / code	controller/monitor.py, anomaly.py, health.py, statemachine.py; tests/test_detection.py
Status / to do	Done.

9. Fault-Tolerant Recovery and Resilience Layer

Plan: content focus	Checkpointing, replication, lineage recovery, failover mechanisms, workload migration, self-healing.
Plan: visuals	Figure 13: Fault Recovery Architecture (+ (b) τ^* vs MTBF plot) Figure 14: Failover and Recovery Workflow
Word count	Target 1200–1800 | current 1019
Owner	26MML0015
What to write	1. Three recovery levels: task (Spark), attempt (retry on same cluster), cluster (migrate). 2. Adaptive checkpointing: Young/Daly τ^* (Eq. 4), expected lost work (Eq. 5), online MTBF estimate (Eq. 6). 3. Stage restoration, progress-aware failover rules (Table 16 in appendix), self-healing, correctness-of-resumption argument.
Cite	[5][20][23][27][39][43]–[49]
Our evidence / code	rds/policy.py, rds/stage.py, controller/jobs.py; tests/test_policy.py, test_controller_sim.py
Status / to do	Add measured τ^* range and checkpoint counts after experiments.

10. Multi-Cluster Coordination and Orchestration Layer

Plan: content focus	Cluster synchronization, workload balancing, orchestration policies, service discovery, coordination protocols.
Plan: visuals	Figure 15: Multi-Cluster Coordination Model Figure 16: Workload Orchestration Framework
Word count	Target 1000–1500 | current 799
Owner	26MML0015
What to write	1. Centralised, asynchronous coordination through shared state (no cluster-to-cluster messages). 2. Cluster registry as service discovery; placement cost J (Eq. 7) with α=0.4, β=0.5, γ=0.1; why a weighted sum rather than a learned scheduler. 3. 5-step coordination protocol; synchronisation & consistency (idempotent writes, fencing); scaling to more clusters.
Cite	[14][28][78][84][86][87][90]
Our evidence / code	controller/orchestrator.py, configs/clusters.yaml; tests (placement)
Status / to do	About 200 words short — add a worked placement example with three clusters, or a paragraph on the effect of the weights.

11. Explainability and Reliability Assessment Layer

Plan: content focus	Explainability of recovery decisions, root-cause analysis, reliability scoring, operational transparency.
Plan: visuals	Figure 17: Explainable Reliability Framework Table 7: Reliability Assessment Indicators Equation: Reliability Score Model (Eq. 8)
Word count	Target 1000–1500 | current 846
Owner	26MML0054
What to write	1. Decision log contents (transition, failover, keep, job end) with per-signal contributions. 2. Root-cause reading of the log; reliability score R (Eq. 8) and its four indicators (Table 7). 3. Worked example of one outage (four log records); limits of additive explanations; operational transparency (API, CLI, Grafana).
Cite	[30][65][74][75][93][94]+
Our evidence / code	controller/decisions.py, reliability.py, api.py
Status / to do	After experiments, paste a real excerpt of decisions.jsonl.

12. Security, Governance, and Compliance Layer

Plan: content focus	Security controls, access management, disaster recovery policies, auditing, governance monitoring, compliance.
Plan: visuals	Figure 18: Security and Governance Architecture Table 8: Governance and Compliance Controls
Word count	Target 800–1200 | current 651
Owner	26MML0047
What to write	1. Be honest: separate IMPLEMENTED controls (private network, storage credentials, decision log, per-job namespaces, provenance markers) from DESIGNED ones (Spark auth, TLS, bucket policies, RBAC, non-root, secrets). 2. Threat model (rogue submission, forged checkpoints, steered failover); access & auditing; DR policies (RTO/RPO); jurisdiction-aware placement.
Cite	[95][112][113][118][119]
Our evidence / code	Table 8 status column.

Status / to do	About 150 words short — optional: actually enable spark.authenticate and move it to 'implemented'.

13. Experimental Setup

Plan: content focus	Datasets, Spark clusters, cloud infrastructure, network environment, benchmark workloads, evaluation configurations.
Plan: visuals	Figure 19: Experimental Testbed Architecture Table 9: Experimental Configuration Details (CORE TABLE)
Word count	Target 700–1200 | current 782
Owner	26MML0006
What to write	1. Host (12 CPUs, 16 GB, Docker/WSL2 12 GB), container limits, Spark 3.5.3 image. 2. Workloads W1–W4 + 2 case studies; 9 fault types (transient vs persistent); 5 configurations B1, B2, B3, A1, P. 3. Procedure: shuffled runs, raw files, runner errors recorded. Update Table 9 if you change repetitions or fault times.
Cite	[19][53][95][96]
Our evidence / code	experiments/configs/full.yaml, chaos/inject.py, results/raw/full/experiment.json
Status / to do	Fill measured clean runtimes and the final fault_at_s values.

14. Evaluation Metrics

Plan: content focus	Fault tolerance metrics, recovery time, availability, throughput, latency, scalability, resource efficiency, reliability indicators.
Plan: visuals	Figure 20: Evaluation Framework Table 10: Metrics Definition Matrix
Word count	Target 500–800 | current 505
Owner	26MML0054
What to write	1. Availability (Eq. 9), recovery time (Eq. 10), overhead (Eq. 11), utilisation (Eq. 1), streaming recovery rule (≥ 85 % for 40 s, as Vogel et al.). 2. Statistics: Mann-Whitney U, Cliff's δ, bootstrap CI; metric validity limitations.
Cite	[29][53][54][94]
Our evidence / code	analysis/metrics.py, analysis/stats.py
Status / to do	Done.

15. Results and Findings

Plan: content focus	Recovery performance, cluster resilience, workload continuity, throughput stability, system efficiency, operational availability.
Plan: visuals	Figure 21: Fault Recovery Performance (CORE) Figure 22: Availability Analysis (CORE) Figure 23: Resource Utilization Trends (CORE) Table 11: Experimental Results Summary (CORE)
Word count	Target 1200–2000 | current 668
Owner	26MML0054
What to write	1. Hypotheses H1–H5 are already written — for each, report the measured value, p-value, Cliff's δ and say SUPPORTED / NOT SUPPORTED. 2. Replace every [[...]] with numbers from results/report (Table 11 CSV, runs_enriched.csv). 3. Copy Figs 21–23 PNGs from results/report into paper/Diagrams with the same names and rebuild.

	4. Negative results are fine — explain them (e.g., overhead higher than expected).
Cite	[10][52][53]
Our evidence / code	analysis/report.py output
Status / to do	BLOCKED until experiments run. Expected to reach ~1200+ words when filled.

16. Comparative Analysis

Plan: content focus	Compare against standard Spark deployments, single-cluster architectures, Kubernetes-based analytics, high-availability platforms.
Plan: visuals	Figure 24: Comparative Radar Chart (CORE) Table 12: Comparative Benchmark Analysis (CORE)
Word count	Target 800–1200 | current 505
Owner	26MML0006
What to write	1. Table 12 upper half (capabilities by design) is done; fill the measured lower half. 2. Ablation logic: B1 vs B2, B1 vs B3, B3 vs A1, A1 vs P — state each measured difference. 3. Kubernetes and published benchmarks: compare qualitatively only (different hardware) — never put Vogel et al.'s numbers in your table.
Cite	[52][53][78][79][106]
Our evidence / code	Table 12 from report.py; Fig 24 radar
Status / to do	BLOCKED until experiments; optional B4 Kubernetes run.

17. Reliability and Business Continuity Assessment

Plan: content focus	Resilience, disaster recovery readiness, business continuity, operational reliability, trustworthiness.
Plan: visuals	Figure 25: Business Continuity Framework Table 13: Reliability and Continuity Assessment Matrix
Word count	Target 700–1000 | current 534
Owner	26MML0047
What to write	1. RTO/RPO for analytics jobs (Fig. 25); design-derived bounds (≈16 s + driver start-up; ≤ τ^* + one stage). 2. Measured columns of Table 13 (max downtime, mean recomputed work); final R of clusters A and B; DR readiness questions; trustworthiness.
Cite	[38][94][95][99]
Our evidence / code	decision log, stage events
Status / to do	Fill measured columns and driver start-up time.

18. Industry Use Cases and Case Study Evaluation

Plan: content focus	Applications in banking, healthcare, telecommunications, manufacturing, smart cities, cybersecurity.
Plan: visuals	Figure 26: Financial Analytics Resilience Architecture Figure 27: Smart Manufacturing Analytics Platform Table 14: Industry Use Cases Matrix
Word count	Target 800–1500 | current 671
Owner	26MML0047
What to write	1. One sub-section per domain with its resilience need and the RMC-Spark mechanism that meets it.

	2. Two REAL case studies: case_fraud (ULB credit card) and case_maintenance (AI4I 2020) — run each with a cluster outage, report completion, restored stages, and AUC equal to the uninterrupted run. 3. Case-study protocol and lessons for adoption are written.
Cite	[120]–[127]
Our evidence / code	controller/workloads.py (case_fraud, case_maintenance), scripts/download_data.py
Status / to do	Run the 2 case studies and fill [[...]].

19. Business Impact Assessment

Plan: content focus	Downtime reduction, resource savings, performance improvement, operational resilience, ROI.
Plan: visuals	Figure 28: Business Impact Dashboard Table 15: Cost-Benefit Analysis
Word count	Target 500–1000 | current 344
Owner	26MML0054
What to write	1. Net benefit model ΔB = N_f (D_base − D_rmc) C_d − C_extra (Eq. 12). 2. D_base, D_rmc, ΔW, O come from experiments; N_f, C_d, C_extra are organisation-specific — state assumptions and their source. 3. Draw Fig. 28 as a sensitivity plot of ΔB over N_f and C_d after experiments.
Cite	[38]
Our evidence / code	analysis results
Status / to do	BLOCKED; will exceed 500 words with the sensitivity discussion.

20. Challenges and Limitations

Plan: content focus	Cross-cluster synchronization issues, latency overhead, network bottlenecks, operational complexity, deployment costs.
Plan: visuals	Figure 29: Challenges and Limitations Roadmap
Word count	Target 600–1000 | current 485
Owner	26MML0015
What to write	1. Controller & MinIO single points of failure; at-least-once stage execution; overhead & geo latency; operational complexity; determinism & data size; threats to validity (single host, small data, 3 reps).
Cite	[53][90][91]
Our evidence / code	README §13 limitations
Status / to do	Add measured overhead numbers; ~100 words to add.

21. Future Research Directions

Plan: content focus	Autonomous recovery systems, AI-driven fault prediction, federated cluster management, self-healing platforms, agentic orchestration.
Plan: visuals	Figure 30: Future Research Roadmap Figure 31: Next-Generation Resilient Analytics Architecture
Word count	Target 600–1200 | current 399
Owner	26MML0054
What to write	1. Predictive signals (failure/workload predictors feeding H); federated clusters; Kubernetes/serverless back-ends; LLM advisory agent over the decision log; learned checkpoint policies; formal verification; energy/cost-aware placement; threshold calibration.

Cite	[35][44][62][66][84][108][109][114]+
Our evidence / code	—
Status / to do	About 200 words short — expand one direction (e.g., Kubernetes back-end) into a paragraph.

22. Conclusion

Plan: content focus	Summarize contributions, findings, practical implications, future opportunities.
Plan: visuals	Figure 32: Research Contribution Framework
Word count	Target 500–800 | current 392
Owner	26MML0015
What to write	1. Restate problem and contributions (Fig. 32); 2–3 measured findings + hypotheses outcome; practical implication; lessons learned; future work.
Cite	—
Our evidence / code	—
Status / to do	Insert findings → will reach 500 words.

References

Plan: content focus	Research literature on Spark, distributed systems, fault tolerance, cloud computing, resiliency engineering, data science platforms.
Plan: visuals	No visuals required
Word count	Target 80–140 (plan); ≥ 30–50 (guidelines) | current 127 refs
Owner	All (26MML0047 lead)
What to write	1. IEEE numeric style, numbered in order of first citation; cite as [1] or [1][3][5] at the end of the sentence (guideline format). 2. All 127 come from OpenAlex records; none retracted; 114 journal / 13 conference; 108 from 2020–2025. 3. Do not add any reference you have not opened; re-run the citation check if you add new ones.
Cite	paper/refs/refs_keyed.json
Our evidence / code	build_paper.py numbers them automatically
Status / to do	Done.

Appendix (Optional)

Plan: content focus	Cluster configurations, recovery algorithms, benchmark workloads, Spark tuning parameters, monitoring rules, supplementary results.
Plan: visuals	Additional figures and tables as required (Table 16: Recovery rules per mode)
Word count	Target 500–2000 | current 146
Owner	26MML0006
What to write	1. Add: full spark-defaults.conf, clusters.yaml, pseudo-code of the control loop and of a resilient stage, per-workload result tables.
Cite	—
Our evidence / code	configs/, docker/, controller/core.py
Status / to do	Optional: expand to ~500 words with pseudo-code + per-workload results.

Part E — Compliance Checklists

Publication statistics (page 5 of the plan)

Component	Recommended	Current manuscript	Status
Total words	10,000–15,000	≈16,600 prose (plan section minimums add to ≈18,900)	Confirm limit with faculty
Figures	25–35	32 (27 drawn + 5 result placeholders)	Replace Figs 21–24, 28 after experiments
Tables	12–15	15 + 1 appendix table	OK
Equations	6–12	12, native Word (editable)	OK
References	80–140	127	OK

Core figures and tables

Item	Status
Fig 6 Resilient Multi-Cluster Spark Architecture (core)	Done (full width)
Fig 21 Fault Recovery Performance	Placeholder — from analysis/report.py
Fig 22 Availability and Uptime Evaluation	Placeholder — from analysis/report.py
Fig 23 Resource Utilization and Cluster Stability	Placeholder — from analysis/report.py
Fig 24 Comparative Benchmark Analysis	Placeholder — from analysis/report.py
Table 3 Literature Review Comparison	Done
Table 6 Monitoring Metrics and Thresholds	Done (exact config values)
Table 9 Experimental Configuration Details	Done (update if config changes)
Table 11 Experimental Results Summary	Structure done — fill numbers
Table 12 Comparative Benchmark Analysis	Capabilities done — fill measured part

Manuscript guidelines

Guideline	How it is met / what to do
Official template, formatting	Manuscript is filled into 3_Consolidated_Template.docx (cover, Index, lists, Contributions, IEEE body)
Diagrams ≥ 600 DPI, separate folder, 'Fig NN Name'	paper/Diagrams/ — all 600 DPI. Guideline asks for draw.io: redraw if your faculty insists (ours are original, script-drawn)
Cite source if a diagram is reused	All diagrams are original — no 'Adapted from' needed
Equations editable, MS Equation Editor, numbered	12 OMML equations (Word equation editor), numbered (1)–(12), referenced in text
30–50 refs min; 80 % IEEE/ACM/Springer/Elsevier; 2020–2025	127 refs; 114 journal; 108 from 2020–2025
Citation format [1] / [1][3][5]	Used throughout
Similarity ≤ 10 %	Text written originally — still run the similarity checker before submission
Deliverables	Manuscript (.docx), Diagram folder, source code (push rmc-spark/ to GitHub), datasets folder (datasets/ + scripts), execution videos (README §11)
Data availability statement	In the manuscript; insert the GitHub link

Part F — Step-by-Step Plan to Finish

1. Start Docker Desktop (12 GB in .wslconfig), then: docker compose build ; docker compose up -d (README Steps 1–3).
2. Generate data: datagen spark-submit (README Step 4). Optional: download_data.py --tlc-months 2024-01 --case-studies, then ingest_local.py.
3. Live demo and video: submit etl_taxi, inject cluster_outage on A, show failover in Grafana and controller.cli decisions (README Step 5, §11).
4. Smoke experiment: experiments/configs/smoke.yaml — expect B1 FAILED, P SUCCEEDED on A>B (Step 6).
5. Calibrate fault_at_s and expected_runtime_s in full.yaml from clean runtimes; run the full matrix (Step 7).
6. Run case studies case_fraud and case_maintenance with a cluster outage (Section XVIII).
7. python -m analysis.report results/raw/full --out results/report (Step 8).
8. Copy Fig 21–24 PNGs into paper/Diagrams (same names); build Fig 28 sensitivity plot; fill every yellow [[...]] in paper/content/*.txt.
9. Fill student names, e-mails, bios, cover names; GitHub link; delete the AUTHOR NOTE in Section XV.
10. Rebuild: python paper/make_figures.py ; python paper/build_paper.py ; open in Word, Ctrl+A then F9 to refresh page numbers.
11. Run the similarity check (≤ 10 %), re-check references if any were added, push code + datasets scripts + results to GitHub, record execution videos.

Team responsibilities

Member	Implementation	Sections	Current words
26MML0015 (PoC)	controller core, jobs, orchestrator, integration	Exec summary, 5, 9, 10, 20, 22	4148
26MML0054	monitoring, anomaly, health, reliability, analysis	Keywords, 8, 11, 14, 15, 19, 21	3765
26MML0006	Docker testbed, jobs, rds library, chaos, experiments	2, 4, 6, 7, 13, 16, Appendix	4479
26MML0047	literature, references, security, datasets, case studies	1, 3, 12, 17, 18, References	4214